데이터 타입과 기초 연산

컴퓨터학부 박한영

목차

01 변수

02 자료형

03 메모리

04 형변환

05 연산자

06 입출력

변수

<변수의 개념>

- 데이터를 저장하기 위해 프로그램에 의해 이름을 할당 받은 메모리 공간
- 프로그램이 실행되는 동안 언제든지 저장된 값이 변경될 수 있는 공간
- C언어에서는 메모리 저장 위치에 따라 전역변수, 지역변수, 동적변수로 나눌 수 있다.

<변수의 생성 규칙>

- 변수명은 알파벳 대소문자, 숫자, 언더바(_)로만 구성된다.
- 변수명은 숫자로 시작할 수 없다.
- 변수의 이름으로 C에서 미리 정의된 키워드는 사용할 수 없다. (e.g. int, char..)
- 변수를 선언할 시 자료형 + 변수명; 의 구조로 선언한다.

```
// 변수 선언
int i = 10;
char c = 'A';
```

자료형

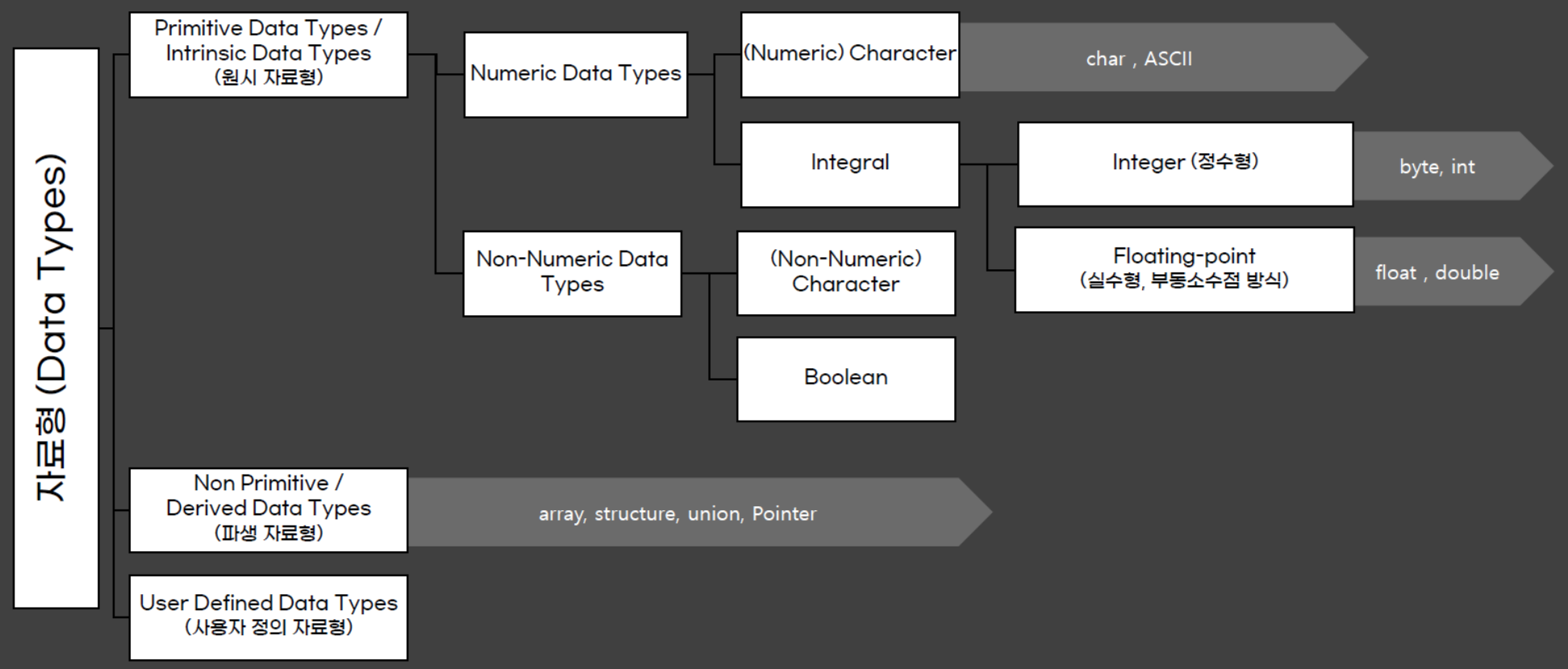
<자료형(Data Type)>: 여러 종류의 데이터를 식별하는 분류

C언어에서 주로 사용되는 자료형

- 정수형: 정수를 표현하는 자료형들. 메모리 크기 순으로 short, int, long, long long이 있음
- 실수형(float): 소수점을 포함하는 실수를 표현하는 자료형들. float, double이 존재함
- 문자형(character): 문자를 표현하는 자료형으로 아스키코드로 저장됨
- 포인터(pointer): 메모리 주소를 저장하는 자료형 (자료형으로 보지 않는 견해도 있음)
- 논리형(boolean): 타 언어에는 존재하나 C언어에서는 자료형으로는 존재하지 않음. 내부적으로는 0이면 False, 나머지는 True로 처리함.

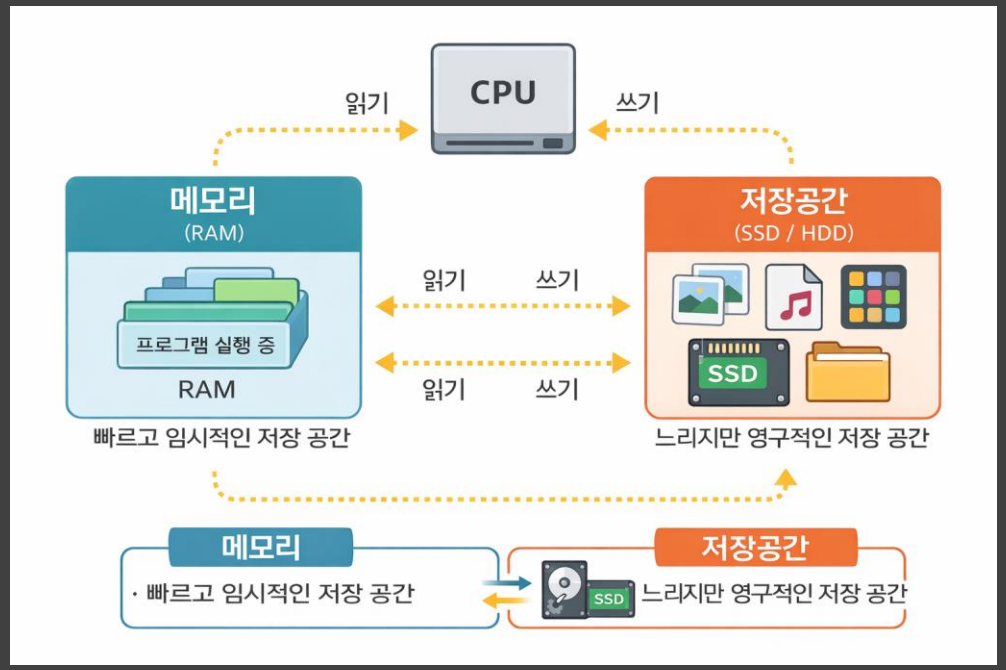
자료형

자료형의 종류들


자료형 (Data Types)
Primitive Data Types / Intrinsic Data Types (원시 자료형)
Numeric Data Types
(Numeric) Character
char , ASCII
Integral
Integer (정수형)
byte, int
Floating-point (실수형, 부동소수점 방식)
float , double
Non-Numeric Data Types
(Non-Numeric) Character
Boolean
Non Primitive / Derived Data Types (파생 자료형)
array, structure, union, Pointer
User Defined Data Types (사용자 정의 자료형)

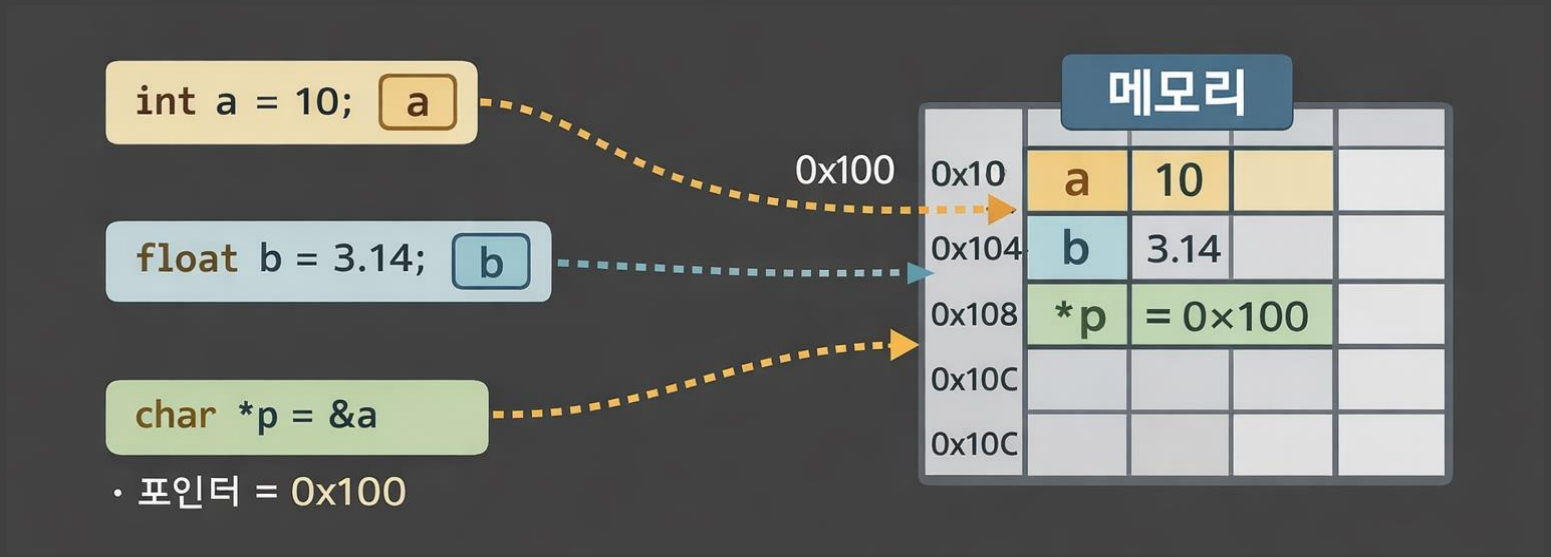
메모리

- 메모리는 컴퓨터에서 데이터를 저장하는 공간
- 프로그램에서 사용하는 모든 데이터는 메모리에 저장됨
- CPU는 메모리에서 데이터를 가져와 연산하고, 그 결과를 다시 메모리에 저장



변수와 메모리

- 변수를 선언하면 메모리의 공간을 할당받음
- 이 때 변수가 선언된 메모리 공간의 위치를 메모리 주소라고 함.
- 변수는 메모리 주소를 기억하고 있다가, 참조가 되면 저장된 곳으로 가서 그곳에 있는 값을 참조



변수와 메모리

- 변수가 저장되는 메모리 영역에 따라 아래와 같이 구분할 수 있음.

자료형과 메모리

- 다양한 자료형들은 컴파일러가 이진수로 변환하여 저장됨.
- 비트(bit): 컴퓨터가 데이터를 처리하기 위해 사용하는 데이터의 최소 단위(0과1)
- 바이트(byte): 8개의 비트로 이루어진 컴퓨터의 기본적인 데이터 저장 단위
- 숫자의 경우 부호의 유무에 따라 singed와 unsigned로 나눌 수 있다.

자료형과 메모리

자료형에 따른 메모리 크기

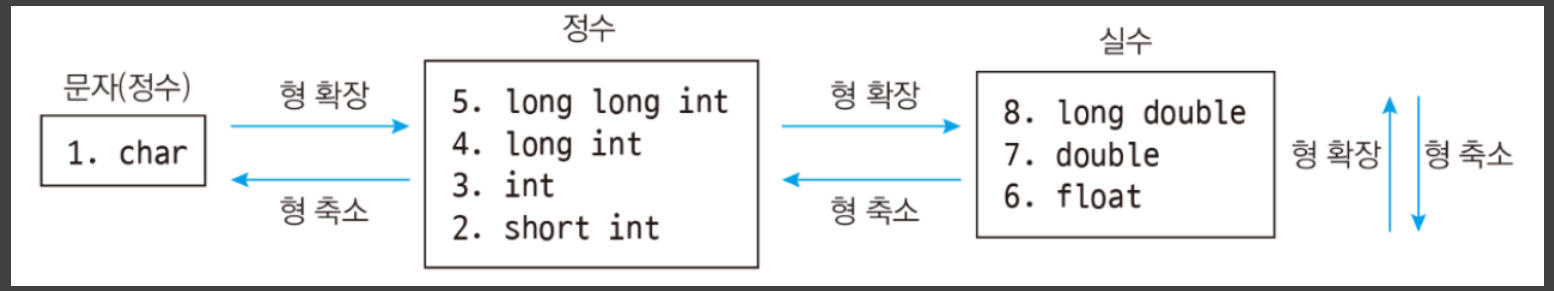
주의! 운영체제마다 크기가 달라질 수 있음!

자료형	메모리 크기	값의 범위
char	1 Bytes	-128 ~ 127
(signed) short	2 Bytes	-32,768 ~ 32,767
Int	4 Bytes	-2,147,483,648 ~ 2,147,483,647
(signed) long	4 Bytes	-2,147,483,648 ~ 2,147,483,647
(signed) long long	8 Byte	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
unsinged char	1 Bytes	0 ~ 255
unsigned short	2 Bytes	0 ~ 65,535
unsigned int	4 Bytes	0 ~ 4,294,967,295
unsigned long	4 Bytes	0 ~ 4,294,967,295
float	4 Bytes	1.2E-38 ~ 3.4E38
double	8 Bytes	2.2E-308 ~ 1.8E308
void	0 Bytes	값이 없다

형 변환

- 형 변환이란? → 어떤 데이터의 자료형을 다른 자료형으로 바꾸는 것!
- 명시적 형변환(Explicit Casting)과 암시적 형변환(Implicit Casting)으로 나뉨

[자료형의 메모리 크기 – 형 확장과 축소]



형 변환

- 명시적 형변환(Explicit Casting): 내가 원하는 시점에 형변환을 하는 것이며, 변환할 자료형을 변수나 값 앞에 작성

```
int a = 5, b = 2;
float c = a / b;
float d = (float)a / b;
printf("%.1f %.1f", c, d);
// 결과: 2.0 2.5
```

- 암시적 형변환(Implicit Casting): 자료형의 우선순위에 따라 데이터의 자료형이 자동적으로 바뀌는 형변환이며, 자료형의 메모리 크기가 클 수록 우선순위가 높다.

```
int a = 3;
double b = 2.0;
double c = a + b;
// a가 float로 형변환
```

연산자

연산자의 종류

구분	연산자
대입 연산자	=
산술 연산자	+, -, *, /, &, ++, --
관계 연산자	<, >, <=, >=, ==, !=
논리 연산자	&&, ||, !
할당 연산자	+=, -=, *=, /=, %= 등...
삼항 연산자	?
비트 연산자	&, |, ~, ^, <<, >>

연산자

<대입 연산자>

- (변수 식별자) = (식)의 구조로 되어 있음
- 좌변에 우변의 값을 할당한다고 생각!

```
int a = 1, b = 0;

b = a++;     // b = 1, a = 2
b = a--;     // b = 1, a = 0

b = ++a;     // b = 2, a = 2;
b = --a;     // b = 0, a = 0;
```

<산술 연산자>

- C언어에서는 사칙연산과 나머지 연산을 지원
- 마찬가지로 좌변에 우변의 값을 할당한다고 생각!
- 단항 연산자: 전위형과 후위형으로 나눌 수 있음

연산자	기능	문법	
+	더하기	a = 5 + 2;	5와 2를 더해 a에 넣는다
-	빼기	a = 5 - 2;	5에서 2를 빼 a에 넣는다.
*	곱하기	a = 5 * 2	5와 2를 곱해 a에 넣는다.
/	나누기	a = 5 / 2	5를 2로 나눈 몫을 a에 넣는다.
%	나머지	a = 5 % 2	5를 2로 나눈 나머지를 a에 넣는다.
++	1을 증가시킨다.	a++	a = a +1
--	1을 감소시킨다.	a--	a = a - 1

연산자

<할당 연산자>: 대입 연산자 + 산술 연산자의 개념

연산자	문법	의미
+=	a += 1	a = a + 1
-=	a -= 2	a = a - 2
*=	a *= 3	a = a * 3
/=	a /= 4	a = a / 4
%=	a %= 5	a = a % 5

연산자

<관계 연산자>

- (좌변) *관계연산자* (우변) 의 구조
- 연산의 결과값은 양변의 자료형과 상관없이 0과1
- 여러 번의 연산은 아래의 논리 연산자와 연계

연산자	기능
<	왼쪽이 오른쪽보다 작으면 1, 크거나 같으면 0
>	왼쪽이 오른쪽보다 크면 1, 작거나 같으면 0
<=	왼쪽이 오른쪽보다 작거나 같으면 1, 크면 0
>=	왼쪽이 오른쪽보다 크거나 같으면 1, 작으면 0
==	두 값이 같으면 1, 다르면 0
!=	두 값이 다르면 1, 같으면 0

<논리 연산자>

- (좌변) *논리연산자* (우변) 의 구조
- 연산의 결과값은 양변의 자료형과 상관없이 0과1
- 괄호를 통해 논리 연산 우선순위 지정 가능

연산자	논리	기능
&&	AND	양쪽이 모두 참이면 참
||	OR	양쪽 중 하나이상이 참이면 참
!	NOT	참이면 거짓, 거짓이면 참

연산자

<비트 연산자>: 비트 단위 연산을 실행하는 연산자

연산자	기능
&	비트단위 AND 연산
|	비트단위 OR 연산
~	비트단위 NOT 연산
^	비트단위 XOR 연산
<<	왼쪽으로 비트 이동
>>	오른쪽으로 비트 이동

<삼항 연산자>

연산자	설명	문법
? :	조건부 연산자. 조건식이 참이면 : 앞의 값을 반환, 거짓이면 : 뒤의 값을 반환	x ? a : b;

```
a = 1010 0101, b = 1001 1001일때

c = a & b;     // c = 1000 0001
c = a | b;     // c = 1011 1101
c = ~a;        // c = 0101 1010
c = a^b;       // c = 0011 1100

a = 0011 1010; 일때

c = a << 1;     // c = 0111 0100
c = a << 2;     // c = 1110 1000

c = a >> 1;     // c = 0001 1101
c = a >> 2;     // c = 0000 1110

int a = 12;

c = a << 1;     // c = 24 (= a * 2)
c = a << 3;     // c = 96 (= a * 8)

c = a >> 1;     // c = 6 (= a / 2)
c = a >> 2;     // c = 3 (= a / 4)
```

연산자

연산자 우선 순위

기호 1	연산 유형	associativity
`[` `]` `(` `)` `.` `->++--` (후위)	식	왼쪽에서 오른쪽
`sizeof` `&` `*` `+` `-` `~` `!++--` (전위)	단항	오른쪽에서 왼쪽
형식 캐스팅	단항	오른쪽에서 왼쪽
`*` `/` `%`	곱하기	왼쪽에서 오른쪽
`+` `-`	더하기	왼쪽에서 오른쪽
`<<` `>>`	비트 시프트	왼쪽에서 오른쪽
`<` `>` `<=` `>=`	관계	왼쪽에서 오른쪽
`==` `!=`	같음	왼쪽에서 오른쪽
`&`	비트 AND	왼쪽에서 오른쪽
`^`	비트 제외 OR	왼쪽에서 오른쪽
`|`	비트 포함 OR	왼쪽에서 오른쪽
`&&`	논리 AND	왼쪽에서 오른쪽
`||`	논리 OR	왼쪽에서 오른쪽
`? :`	조건식	오른쪽에서 왼쪽
`=` `*=` `/=` `%=+=` `-=` `<<=` `>>=` `&=^=` `|=`	단순 및 복합 할당 2	오른쪽에서 왼쪽
`,`	순차적 계산	왼쪽에서 오른쪽

출력

<printf() 함수>

- C언어의 표준 출력 함수
- 출력할 데이터를 서식 지정자를 통해 원하는 서식으로 출력
- 서식 문자열 내에 특수문자와 서식 문자가 포함됨

```
int printf(const char * restrict format, ...);

ex) printf("%d", n);
    printf("Hello World!")
```

출력

출력 시 문자열 특수문자표

특수문자	의미
₩₩	화면에 역슬래시(₩) 출력
₩'	작은 따옴표의 출력
₩"	큰 따옴표의 출력
₩?	물음표의 출력
₩a	경고음
₩b	백스페이스
₩f	Form feed
₩n	return
₩r	carriage return
₩t	수평 탭
₩v	수직 탭

입력

<scanf() 함수>

- C언어의 표준 입력 함수
- 입력할 데이터를 서식 지정자를 통해 원하는 서식으로 받아옴
- 서식 문자열 내에 특수문자와 서식 문자가 포함됨
- scanf는 읽은 값을 어디에 저장할 지 알아야 하므로 변수의 주소가 필요함

```
int scanf(const char * restrict format, ...);

ex) scanf("%d", %n);
```

입출력

서식문자표 및 특용/특수문자표

종류		문자	자료형	비고
정수	10진수	%hi	signed short	부호있는 2바이트 크기의 정수
		%hu	unsigned short	부호없는 2바이트 크기의 정수
		%d	signed int	운영체제마다 다름/보통 4바이트
		%u	unsigned int	운영체제마다 다름/보통 4바이트
		%li	signed long	부호있는 4바이트 크기의 정수
		%lu	unsigned long	부호없는 4바이트 크기의 정수
		%lli	signed long long	부호있는 8바이트 크기의 정수
		%llu	unsigned long long	부호있는 8바이트 크기의 정수
	8진수	%o	unsigned int	8진수 정수 출력
	16진수	%x	unsigned int	16진수 정수 소문자 출력
		%X	unsigned int	16진수 정수 대문자 출력
실수		%f	float	4바이트 크기의 10진수 실수
		%lf	double	8바이트 크기의 10진수 실수
		%llf	long double	12바이트 크기의 10진수 실수
		%e	float , double	부동소수점 e(소문자) 방식 실수
		%E	float , double	부동소수점 E(대문자) 방식 실수
		%g	float , double	경우에 따라 %f와 %e
		%G	float , double	경우에 따라 %f와 %e
문자		%c	char	문자 출력
문자열		%s	char*,char[]	문자열 출력
포인터		%p	void*	포인터 주소값 출력

특용문자	출력	설명
\\	\	특수문자 표현할 때 사용되는 문자 \ 출력
\"	"	문자열상수 표현할 때 사용되는 문자 " 출력
\'	'	문자상수 표현할 때 사용되는 문자 ' 출력
%%	%	서식지정 표현할 때 사용되는 문자 % 출력

특수문자	동작	설명
\n	new line	다음줄 첫칼럼위치로 커서이동
\t	tab	탭위치로 커서이동
\b	back	뒤로 한칼럼 커서이동
\r	carriage return	현재라인의 첫칼럼으로 커서이동
\v	vertical tab	수직 탭효과, 프린터에서 여러라인 이동
\f	form feed	프린팅시 현재 종이밀어내고, 다음 장의 첫라인으로

THANK YOU!